

Nombre: Luis Antonio Cutzal Chali

C

Carné: 201700841

MANUAL TECNICO

Proyecto 1

Ingeniero Kevin Lajpop

Auxiliar: Sandra Giménez

Fecha de Entrega: 05/09/2021

Descripción general de la solución:

De acuerdo con el enunciado del Proyecto 1, se creo una herramienta el cual servirá de apoyo a los tutores académicos en el análisis del código fuente de los diferentes proyectos desarrollados por los estudiantes con el fin de agilizar los procesos de búsqueda de copias.

Se utilizaron diversas librerías, métodos y clases para crear la solución óptima esperada, ya sea para poder ver gráficamente las puntuaciones que se leen en el archivo de entrada para una mayor comprensión de las posibles copias.

Dichas librerías, métodos y clases se especifican más adelante del manual.

Requerimientos mínimos del entorno del desarrollo:

1. Tener instalado NetBeans 8.2 o Apache NetBeans 12.4, esto para poder ejecutar la herramienta, en caso de no tener alguno de los 2 IDE's deberá descargarlo desde la página oficial.
2. Tener instalado JDK de Oracle el cual sirve para los programas en lenguaje Java.
3. Tener descargado los paquetes: java-cup-11b y java-cup-11b-runtime además de haberlos cargado en la aplicación, dichos archivos son para los archivos sintácticos del FCA y JS.
4. Tener descargado el paquete: jflex-1.6.1 además de haberlos cargado en la aplicación, el cual es para los archivos léxicos del FCA y JS.
5. Tener descargado los paquetes de: jfreechart-1.0.1 y jfreechart-1.5.3, haberlos cargados a la aplicación, dichos archivos sirven para generar las gráficas de Barras, Pie y Lineales de los reportes.
6. Tener instalado el sistema operativo Windows 10, cualquier versión de Linux o Mac.
7. Tener como mínimo 4 GB de memoria RAM en su computador.

Diccionario de Clases:

En la herramienta se encuentran diversos tipos de clases, una de ellas es la parte del main el cual ejecuta la ventana de la aplicación.

Ventana.java: Entorno grafico de la aplicación.

error.java: Clase utilizada para los errores que puede tener la lectura de un archivo FCA o JavaScript.

Variables.java: Clase utilizada para la producción de los DATOS de la Gramática del AFC.

Nodo.java: Clase utilizada para crear los diversos nodos utilizados en la Gramática JavaScript.

Comparar.java: Clase utilizada para comparar obtener las rutas de los archivos a comparar.

Archivo.java: Clase utilizada para obtener de los datos de los archivos a comparar.

Valor.java: Clase utilizada para saber los valores utilizados por las Gráficas obtenidos de la Gramática del FCA.

GraficaPie.java: Clase utilizada para generar las Gráficas de Pie.

GraficaLineas.java: Clase utilizada para genera las Gráficas de Líneas.

GraficaBarras.java: Clase utilizada para generar las Gráficas de Barras.

Características.java: Clase utilizada en el archivo **Analizador_Sintactico.cup**, para devolver los valores utilizados para generar las diversas Graficas de la herramienta.

GeneradorSintactico2.java: Generador de la clase **ASintactico.cup**.

GeneradorLexico2.java: Generador de la clase **ALexico.jflex**.

GenSintactico.java: Generador de la clase **Analizador_Sintactico.cup**.

GenLexico.java: Generador de la clase **Analizador_Lexico.jflex**.

SintacticoJS.java: Clase que contiene todas las producciones en lenguaje java de los archivos **ASintactico.cup** y **Analizador_Sintactico.cup**.

Símbolos.java: Clase que contiene todos los símbolos de las gramáticas, dicha clase es generada por **GenSintactico.java** y **GeneradorSintactico2.java**.

ASintactico.cup: Clase para los tokens, lexemas, palabras reservadas, identificadores, símbolos de la gramática de los archivos JavaScript.

ALexico.jflex: Clase específica para los tokens, lexemas, palabras reservadas, identificadores, símbolos de la gramática de los archivos JavaScript.

ALexico.java: Clase específica para el analizador léxico de la gramática FCA, dicha clase es generada al ejecutar la clase **GeneradorLexico2.java**.

Analizador_Sintactico.cup: Clase para las producciones, no terminales y terminales de la gramática de los archivos FCA.

Analizador_Lexico.jflex: Clase para los tokens, lexemas, palabras reservadas, identificadores, símbolos de la gramática de los archivos con extensión fca.

Analizador_Lexico.java: Clase específica para el analizador léxico de la gramática FCA, dicha clase es generada al ejecutar la clase **GenLexico.java**.

Diccionario de Métodos:

Se crearon diversos métodos o funciones para que el sistema pueda ser creado y ejecutado de una buena manera, se enlistaran los métodos más importantes de la aplicación.

Evento Ejecutar: Al presionar dicha opción en la Ventana, ejecutara todas las instrucciones previamente cargadas en el apartado del `jQuery` de la herramienta, comenzara a ejecutar las instrucciones para leer las gramáticas, comparar archivos y generar graficas.

Evento Reporte Errores: Generará un archivo HTML con los errores (si es que los tiene), de la lectura de los archivos que se han cargado en la aplicación.

Evento Reporte Estadístico: Generará un archivo el cual contendrá un resumen de las graficas y tablas que genera la aplicación.

Evento Reporte Tokens: Generara un archivo HTML con todos los tokens reconocidos por las 2 gramáticas creadas para el sistema.

encontrar: Método que recibe un Nodo y una lista de tipo string de variables, el cual sirve para manipular de una forma más fácil los datos de la gramática de JavaScript, introducir dichos datos en otras listas para luego utilizarlas en los métodos de comparación de clases y variables.

Comparamos: Método que recibe 2 atributos de tipo String los cuales son las rutas de los archivos que tenemos que comparar, dicho método funciona solo si 2 archivos se llaman de la misma manera, caso contrario no funcionara, en este método es donde se manipulan los datos mencionados en el método anterior con el fin de saber cuales son las variables, clases o comentarios repetidos el cual también maneja una lista para los errores que se encuentren a lo largo de la ejecución de la aplicación.

variables_repetidas: Dicho método es para mostrar en la consola de la aplicación las variables repetidas que se encontraron al momento de ejecutar las gramáticas y comprar archivos.

comentariosrepetidos: Este método es como el anterior, solo que, en lugar de mostrar variables, muestra comentarios repetidos.

ClasesRepetidas: Su función es llevar un control de las clases que se van repitiendo dentro de los archivos JavaScript a analizar en busca de copias.

MetodosRepetidos: Su función es llevar un control de los métodos repetidos dentro de los archivos JavaScript a analizar.

ReporteErrores: Crea el archivo HTML de los errores anteriormente mencionado, el cual muestra el tipo de error, el valor, la fila, la columna y el nombre del archivo el cual está dicho error.

ReporteTokens: Es como el método anterior, solo que este genera un archivo con todos los tokens que se han reconocido por las gramáticas.

generar_graficaBarras: Este método es el que procede a graficar el archivo de barras, teniendo en cuenta los diversos valores que este necesita, además de utilizar las librerías `import org.jfree.chart.JFreeChart;`, `import org.jfree.chart.plot.PlotOrientation;` y `import org.jfree.data.category.DefaultCategoryDataset;` los cuales son necesarios para generar la gráfica de barras.

generar_graficaLineas: Este método es como el anterior, pero con la ligera diferencia de los atributos a utilizar y la ejecución de una instrucción para generar líneas en lugar de barras el cual es `JFreeChart chart = ChartFactory.createXYLineChart();`, el cual recibe dentro de los paréntesis la información de la grafica como lo es: **titulo de la gráfica, nombre del archivo y la puntuación.**

generar_graficaPie: Este método es como los dos anteriores, pero a diferencia de las otras gráficas, esta genera una gráfica de pie, utilizando la función `JFreeChart grafico= ChartFactory.createPieChart();`, el cual recibe dentro de los paréntesis: **titulo de la gráfica, y los datos necesarios para graficar**

Archivo: Este método recibe como parámetros el nombre del archivo, y luego diversas listas ya sea de variables, comentarios, clases, métodos, entre otras, con el fin de llevar el control de los datos repetidos que se encuentran en los archivos a comparar, dicho método es invocado y utilizado en el método de **Comparamos, MetodosRepetidos, ClasesRepetidas, comentariosrepetidos y variables_repetidas**